

第 12 章 异常处理

郑 莉　清华大学

教材：C++语言程序设计（第5版） 郑莉　清华大学出版社

目录

- 异常处理的基本思想
- C++异常处理的实现
- 异常处理中的构造与析构
- 标准程序库异常处理
- 小结

异常处理的思想与程序实现

什么是异常和异常处理

- 异常就是程序在运行过程中出现问题
- “异常”问题并不经常出现
- 异常处理技术使我们可以写程序解决出现的异常问题
- 很多情况下，处理了异常就能使程序像没发生过问题一样继续执行
- 严重的问题也许使程序不能继续执行，使用异常处理可以用可控的方式对用户发出警示，然后再终止程序
- 本章讲的异常处理技术，使我们能够编写更健壮、更具容错性的程序，能处理发生的问题使程序继续执行，或者优雅地终止

应该如何处理异常的思考

- 是否应该频繁检测条件以决定程序接下去该如何执行?
- 请看下面这段伪代码:

```
执行一项任务
如果前序任务执行不正确
    执行错误处理
执行下一项任务
如果前序任务执行不正确
    执行错误处理
...
```

- 上面的伪代码中，每执行一项任务就要检测一下成功与否，不成功便立刻进行错误处理，成功则继续进行下一项任务
- 将程序逻辑与错误处理逻辑混在一起，使得程序的可读性、可修改性、可维护性差，难以调试，特别是大型程序

应该如何处理异常的思考（续）

- 异常处理技术使我们可以将错误处理代码从主逻辑中移出，使程序更清晰，能增强程序的可维护性
- 我们可以有选择地处理异常：所有异常、某种特定类型的所有异常、一组相关类型的所有异常（例如属于某个类继承层次体系的异常）
- 这种灵活性，减小了错误被忽略的可能性，从而使程序更加健壮
- C++的异常处理机制使我们从项目一开始就很容易地设计异常处理

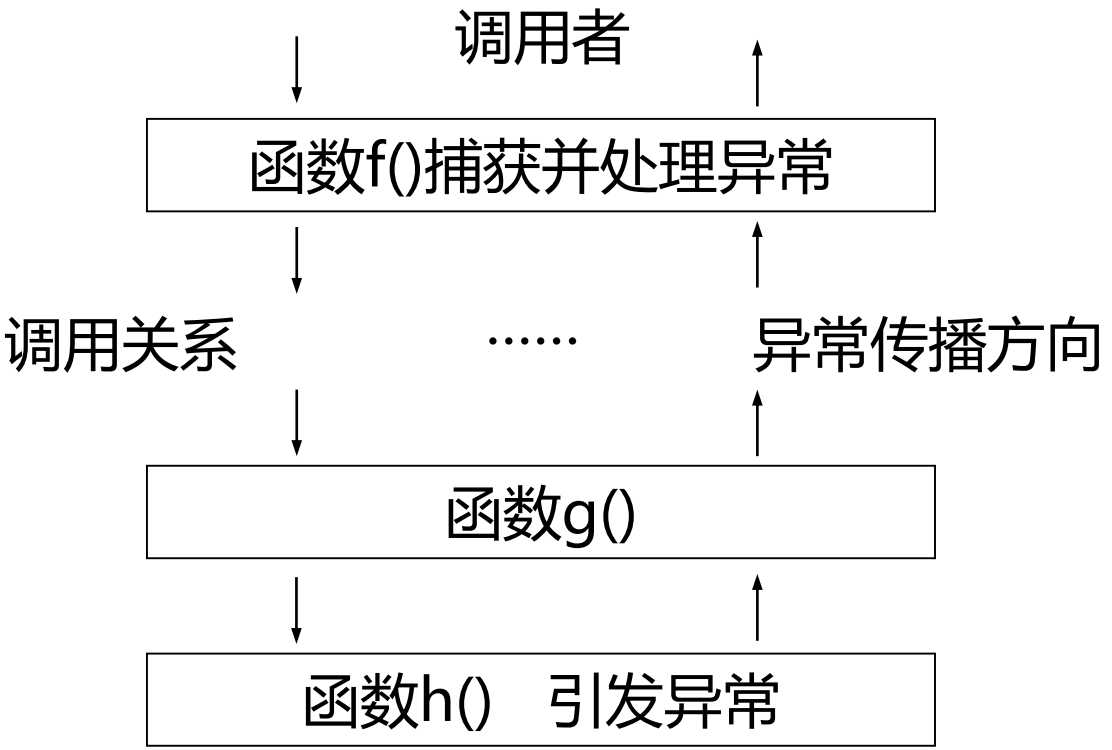
何时使用异常处理

- 异常处理用于处理同步错误，也就是语句执行时发生的错误
- 常见的适用异常处理的错误有：数组下标越界、算术溢出、被零除、函数参数不合法、内存不够引起的内存分配不成功
- 异常处理不是用于处理异步事件相关的错误的，例如：磁盘读写结束、网络信息到达、点击鼠标或键盘，这些与程序的控制流程并行、独立。
- 异常处理机制还适用于处理程序与其他软件元素，例如成员函数、构造函数、析构函数、类交互时出现的问题。
  - 这些软件元素通常并不在内部处理错误，而是使用异常提示程序发生了问题
  - 这就使得每个应用程序可以实现个性化的错误处理

何时使用异常处理（续）

- 复杂的应用程序通常由预定义的软件组件，和调用预定义组件的应用程序特有的软件组件构成
- 当预定义组件遇到异常问题时，需要一种机制将问题通知给调用者，预定义组件无法事先预见到不同的应用程序打算怎样处理异常问题

异常处理的基本思想



异常处理的语法

- 抛掷异常的程序段

```
……
throw 表达式;
……
```

- 捕获并处理异常的程序段

```
try
   复合语句                    保护段
catch（异常声明）
   复合语句                    异常处理程序
catch（异常声明）
   复合语句
   …
```

- 若有异常则通过throw创建一个异常对象并抛掷
- 将可能抛出异常的程序段嵌在try块之中。通过正常的顺序执行到达try语句，然后执行try块内的保护段
- 如果在保护段执行期间没有引起异常，那么跟在try块后的catch子句就不执行。程序从try块后的最后一个catch子句后面的语句继续执行
- catch子句按其在try块后出现的顺序被检查。匹配的catch子句将捕获并处理异常或继续抛掷异常，或部分处理然后继续抛掷。
- 如果匹配的处理器未找到，则库函数terminate将被自动调用，其默认是调用abort终止程序。

例12-1处理除零异常

```
//12_1.cpp
#include <iostream>
using namespace std;
int divide(int x, int y) {
        if (y == 0)
                throw x;
        return x / y;
}
int main() {
        try {
                cout << "5 / 2 = " << divide(5, 2) << endl;
                cout << "8 / 0 = " << divide(8, 0) << endl;
                cout << "7 / 1 = " << divide(7, 1) << endl;
        } catch (int e) {
                cout << e << " is divided by zero!" << endl;
        }
        cout << "That is ok." << endl;
        return 0;
}
```

运行结果：

```
5 / 2 = 2
8 is divided by zero!
That is ok.
```

异常声明（异常规范，Exception specifications）

一个函数显式声明可能抛出的异常，有利于函数的调用者为异常处理做好准备

异常声明（异常规范，Exception specifications）

- 可以在函数的声明中列出这个函数可能抛掷的所有异常类型。
  - 例如：
    ```
    void fun() throw(A, B, C, D);
    ```
- 若无异常接口声明，则此函数可以抛掷任何类型的异常。
- 不抛掷任何类型异常的函数声明如下：

```
void fun() throw();
```

或：`void fun() noexcept;`

noexcept 异常说明

- 对明确不会抛出异常的函数使用noexcept说明符修饰
- 声明方式：返回值类型 func(形参列表) noexcept;
  - 异常处理使编译和运行时有额外开销，省去异常处理可优化加速调用
  - 需保持该函数内部调用函数和定义语句均不会抛出异常的一致性
- 配套有noexcept运算符，可判断函数是否使用了noexcept说明

例： void f() noexcept;
noexcept(f()); //返回true，因为f 有noexcept说明

慎用异常声明的情况

- 对于带类型参数的函数模板,要尽量避免使用exception specifications,因为不同类型对于相同行为的定义不同,抛出的异常也就不同,因而函数模板很难或不可能确定它具现化的函数实体所可能抛出的异常
- 使用回调(callback)函数时
- 系统可能抛出的异常

异常处理中的构造与析构

自动的析构

- 找到一个匹配的catch异常处理后
  - 初始化异常参数。
  - 将从对应的try块开始到异常被抛掷处之间构造（且尚未析构）的所有自动对象进行析构。
  - 从最后一个catch处理之后开始恢复执行。

例12-2 带析构语义的类的C++异常处理

```
//12_2.cpp
#include <iostream>
#include <string>
using namespace std;
class MyException {
public:
        MyException(const string &message) : message(message) {}
        ~MyException() {}
        const string &getMessage() const { return message; }
private:
        string message;
};

class Demo {
public:
        Demo() { cout << "Constructor of Demo" << endl; }
        ~Demo() { cout << "Destructor of Demo" << endl; }
};
```

例12-2 带析构语义的类的C++异常处理

```
void func() throw (MyException) {
    Demo d;
    cout << "Throw MyException in func()" << endl;
    throw MyException("exception thrown by func()");
}

int main() {
    cout << "In main function" << endl;
    try {
        func();
    } catch (MyException& e) {
        cout << "Caught an exception: " << e.getMessage() << endl;
    }
    cout << "Resume the execution of main()" << endl;
    return 0;
}
```

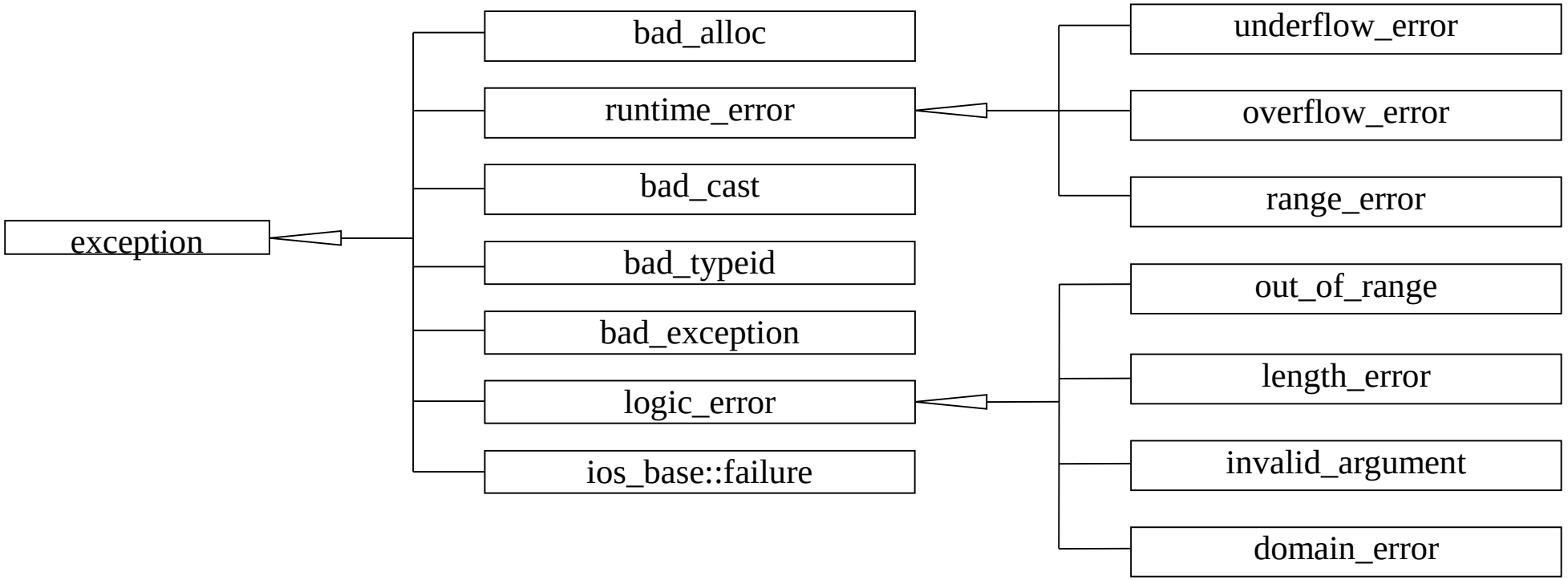
例12-2 带析构语义的类的C++异常处理

运行结果：

```
In main function
Constructor of Demo
Throw MyException in func()
Destructor of Demo
Caught an exception: exception thrown by func()
Resume the execution of main()
```

标准程序库异常处理

标准异常类的继承关系


exception
bad_alloc
runtime_error
bad_cast
bad_typeid
bad_exception
logic_error
ios_base::failure
underflow_error
overflow_error
range_error
out_of_range
length_error
invalid_argument
domain_error

C++标准库各种异常类所代表的异常

异常类	头文件	异常的含义
bad_alloc	exception	用new动态分配空间失败
bad_cast	new	执行dynamic_cast失败（dynamic_cast参见8.7.2节）
bad_typeid	typeinfo	对某个空指针p执行typeid(*p)（typeid参见8.7.2节）
bad_exception	typeinfo	当某个函数fun()因在执行过程中抛出了异常声明所不允许的异常而调用unexpected()函数时，若unexpected()函数又一次抛出了fun()的异常声明所不允许的异常，且fun()的异常声明列表中有bad_exception，则会有一个bad_exception异常在fun()的调用点被抛出
ios_base::failure	ios	用来表示C++的输入输出流执行过程中发生的错误
underflow_error	stdexcept	算术运算时向下溢出
overflow_error	stdexcept	算术运算时向上溢出
range_error	stdexcept	内部计算时发生作用域的错误
out_of_range	stdexcept	表示一个参数值不在允许的范围之内
length_error	stdexcept	尝试创建一个长度超过最大允许值的对象
invalid_argument	stdexcept	表示向函数传入无效参数
domain_error	stdexcept	执行一段程序所需要的先决条件不满足

标准异常类的基础

- exception：标准程序库异常类的公共基类
- logic_error表示可以在程序中被预先检测到的异常
  - 如果小心地编写程序，这类异常能够避免
- runtime_error表示难以被预先检测的异常

例12-3 三角形面积计算

- 编写一个计算三角形面积的函数，函数的参数为三角形三边边长*a*、*b*、*c*，可以用Heron公式计算：
- 设 $p=\frac{a+b+c}{2}$，则三角形面积 $S=\sqrt{p(p-a)(p-b)(p-c)}$

例12-3 三角形面积计算

```
//12_3.cpp
#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;
//给出三角形三边长，计算三角形面积
double area(double a, double b, double c)  throw (invalid_argument)
{
   //判断三角形边长是否为正
   if (a <= 0 || b <= 0 || c <= 0)
     throw invalid_argument("the side length should be positive");
   //判断三边长是否满足三角不等式
   if (a + b <= c || b + c <= a || c + a <= b)
     throw invalid_argument("the side length should fit the triangle inequation");
   //由Heron公式计算三角形面积
   double s = (a + b + c) / 2;
   return sqrt(s * (s - a) * (s - b) * (s - c));
}
```

例12-3 三角形面积计算

```
int main() {
        double a, b, c;         //三角形三边长
        cout << "Please input the side lengths of a triangle: ";
        cin >> a >> b >> c;
        try {
                double s = area(a, b, c);   //尝试计算三角形面积
                cout << "Area: " << s << endl;
        } catch (exception &e) {
                cout << "Error: " << e.what() << endl;
        }
        return 0;
}
```

例12-3 三角形面积计算

- 运行结果1：

Please input the side lengths of a triangle: 3 4 5
Area: 6

- 运行结果2：

Please input the side lengths of a triangle: 0 5 5
Error: the side length should be positive

- 运行结果2：

Please input the side lengths of a triangle: 1 2 4
Error: the side length should fit the triangle inequation

关于是否使用异常的争议（引自“知乎”）

- 异常的好处很明显：
  - 异常不能被忽略。
  - 简化代码。异常减少对错误码的层层传递和检查，使得正常代码的结构看起来更清晰。每个错误码的处理都要写一堆 if xxx else yyy 确实很难受，也很干扰正常逻辑。
  - 异常可以让构造函数更加方便地报错。
- 异常带来的问题也很多：
  - 保证代码异常安全，这对开发者的挑战很大。
  - 引入异常，也就引入了一个隐含的执行路径。很多时候，你根本不知道一个函数会不会抛出异常，可能抛出哪些异常。
  - C++ 异常不带调用栈信息，当在外层捕获到下层自动传播来异常时，现场已经没了。这时候反而可能是手动向上层层传播的错误码+错误日志更有利于定位到真正的问题。
  - 性能。
  - 由于需要 stack unwind，二进制代码文件的大小会比较大。
- 小结
  - 错误码和异常是两种不同的错误处理风格，使用错误码或异常其实各有优劣。C++ 没有强制开发者只能使用其中一种，重要的是在团队和项目中保持统一。个人认为，在一般团队中，要用好异常还是有一定挑战的。

小结

- 主要内容
  - 异常处理的基本思想、C++异常处理的实现、异常处理中的构造与析构
- 达到的目标
  - 简单了解C++的异常处理机制